

```
const nums = [1, 2, 3];  
const more = [0, ...nums, 4, 5];  
console.log(more); // [0, 1, 2, 3, 4, 5]
```

Spread Operator

- ⇒ expands / unpack elements
- ⇒ It takes arrays or object and spread them out piece by piece.

Where to use

Function Calls

```
function greet(a, b, c) {  
  console.log(a, b, c);  
}
```

```
const arr = [1, 2, 3];  
greet(...arr);
```

Array Literals

```
const arr1 = [1, 2];
```

```
const arr2 = [3, 4];
```

```
const combined = [...arr1, ...arr2];
```

```
console.log(combined);
```



```
const user = {name: "Ali", age: 20}
const updated = {...user, age: 21, city: "FSD"}
console.log(updated);
```

```
// {name: "Ali",
//   age: 21,
//   city: "FSD"}
```

Object Literals

```
const obj1 = {a: 1, b: 2}
```

```
d const obj2 = {c: 3}
```

```
const merged = {...obj1, ...obj2}
```

```
console.log(merged);
```

Imp rules about Spread

1. → Only iterable values (array, strings, sets, maps) can be spread in arrays and function calls.

```
const obj = {a: 1}
```

```
const arr = [...obj]
```

// Error (not iterable)

2. Array can be spread into object
`const arr = [10, 20, 30];`
`console.log({ ...arr });`

// { 0: 10, 1: 20, 2: 30 }

3. Primitive behave different in objects.

`const test = { ...true, ..."hi", ...4 }`
`console.log(test);`

// { 0: "h", 1: "i" }

Copy Array/Objects

`const arr1 = [1, 2, 3];`

`const arr2 = [...arr1];`

// shallow copy


```
const obj1 = {x:1, y:2}
```

```
const obj2 = { ...obj1 };
```

// Shallow copy

Concatenate arrays

```
let arr1 = [1, 2];
```

```
let arr2 = [3, 4];
```

```
let combined = [...arr1, ...arr2];
```

```
42}; console.log(combined);
```

// [1, 2, 3, 4]

Condition spreading of Array

```
const isSummer = false;
```

```
const fruits = ["apple", "banana",  
... (isSummer ? ["mango"]; [])];
```

// ["apple", "banana"]

conditional object properties

```
const isSummer = true;
```

```
const prices = { apple: 10,
```

```
... (isSummer & { mango: 30 }) };
```

```
// { apple: 10, mango: 30 }
```

⇒ Spread only do shallow copy.

Rest Operator

1. When you don't know how many arguments a function will get use Rest.

```
function sum(...numbers) {  
  return numbers.reduce((a, b) =>  
    a + b, 0);  
}
```



```
console.log(1, 2));  
console.log(sum(1, 2, 3, 4, 5));
```

or

```
console.log(sum([1, 2, 3, 4]));
```

Mix with normal parameter

```
function greet(greeting, ...names){  
  return `${greeting},  
    ${names.join(" and ")}!`;  
}
```

❗ Rest must be last parameter

In Array Destructuring

```
const colors = ["red", "green", "blue",  
  "yellow"];
```

```
⇒ const [first, second, ...other] = colors
```


Rest in Object Destructuring

```
const person = {  
  name: "John",  
  age: 30,  
  city: "New York",  
  occupation: "Engineer"  
};
```

```
const {name, age, -- details} = person  
console.log(name);  
console.log(age);  
console.log(details);
```

```
// John
```

```
// 30
```

```
// {city: "New York",  
  occupation: "Engineer"}
```


Ternary Operator

⇒ The ternary operator `?:` is a shortcut for if-else.

⇒ It decide b/w two values based on a condition.

Syntax:

condition ? expressionIf True :
expressionIf False

Example:

```
let age = 18;
```

```
let canVote = age >= 18;
```

```
"Yes, you can vote" : "No, you can't"
```

```
console.log(canVote);
```


Ternary

- Short and concise (one line)
- Returns a value directly
- Good for simple condition

If - Else

- Verbose (multi-line)
- Does not return value unless inside return
- Better for complex logic

When to use: for simple logic

When not to use: ~~for~~ when logic is complex